Sliding Window Algorithm in Java

A practical technique for solving array and string problems efficiently

Purpose of these notes
Sliding window maintains information about a continuous section of an array or string. Instead of recalculating the result for every possible section, we update the current window as it moves. This often reduces a brute-force solution from $O(n \times k)$ or $O(n^2)$ to $O(n)$.

1. The basic idea

Imagine a window of fixed size moving across an array. If the array is [2, 1, 5, 1, 3, 2] and the window size is 3, the windows are [2, 1, 5], [1, 5, 1], [5, 1, 3], and [1, 3, 2]. When the window moves one step, the leftmost element leaves and the next element enters.

The key observation is that neighbouring windows overlap. We should not add every element again. Keep the current sum (or other useful state), subtract what leaves, and add what enters.

Two common forms

Fixed-size window: the number of elements stays constant, such as finding the maximum sum of any 3 consecutive elements.

Variable-size window: the window grows or shrinks to satisfy a condition, such as finding the shortest subarray whose sum is at least a target. For the common sum-based pattern, this logic relies on non-negative values; negative values can invalidate it.

When to consider sliding window

Look for phrases such as *contiguous subarray*, *substring*, *consecutive elements*, *longest*, *shortest*, or *maximum/minimum over every window*. It generally does not apply to arbitrary non-contiguous selections.

Before coding, identify what the window represents, what information must be maintained, and when the left or right boundary should move.

2. Fixed-size sliding window

For a window of size k, first calculate the sum of the first k elements. Then move one position at a time. Subtract the element leaving at the left, and add the new element entering at the right.

Pattern to remember

```
sum = sum of the first k elements
best = sum

for right from k to n - 1:
    sum = sum - arr[right - k] + arr[right]
    best = max(best, sum)
```

The expression `arr[right - k]` is the element leaving the window; `arr[right]` is the new element entering. At each iteration, the sum belongs to exactly k consecutive elements.

Java method

```
static long maxWindowSum(int[] arr, int k) {
    if (arr == null || k <= 0 || k > arr.length) {
        throw new IllegalArgumentException("Invalid window size");
    }

    long sum = 0;
    for (int i = 0; i < k; i++) {
        sum += arr[i];
    }
    long best = sum;

    for (int right = k; right < arr.length; right++) {
        sum = sum - arr[right - k] + arr[right];
        best = Math.max(best, sum);
    }
    return best;
}
```

Time: O(n), because each element is added or removed a constant number of times. Extra space: O(1), because only a few variables are maintained. Using long helps avoid integer overflow when values or array sizes are large.

Manual trace

Window	Elements	Sum
1	[2, 1, 5]	8
2	[1, 5, 1]	7
3	[5, 1, 3]	9
4	[1, 3, 2]	6

The maximum is 9, from [5, 1, 3]. We reuse the previous sum instead of recalculating every window from scratch.

3. Problem 1 — Maximum sum of a subarray of size k

Problem: Given an integer array and a positive integer k, return the largest sum among all contiguous subarrays containing exactly k elements.

Example input: arr = [2, 1, 5, 1, 3, 2], k = 3
Expected output: 9

Approach

1. Add the first k elements: 2 + 1 + 5 = 8.
2. Slide right: remove 2 and add 1, so the sum becomes 8 – 2 + 1 = 7.
3. Slide again: remove 1 and add 3, so the sum becomes 7 – 1 + 3 = 9.
4. Slide again: remove 5 and add 2, so the sum becomes 9 – 5 + 2 = 6.
5. Keep the largest sum encountered: 9.

Complete Java program

```
public class MaximumWindowSum {
    static long maxWindowSum(int[] arr, int k) {
        if (arr == null || k <= 0 || k > arr.length) {
            throw new IllegalArgumentException("Invalid input");
        }
        long windowSum = 0;
        for (int i = 0; i < k; i++) {
            windowSum += arr[i];
        }
        long maxSum = windowSum;

        for (int right = k; right < arr.length; right++) {
            windowSum -= arr[right - k];
            windowSum += arr[right];
            maxSum = Math.max(maxSum, windowSum);
        }
        return maxSum;
    }

    public static void main(String[] args) {
        int[] arr = {2, 1, 5, 1, 3, 2};
        int k = 3;
        System.out.println(maxWindowSum(arr, k));
    }
}
```

Output: 9

Complexity and interview point

Time: O(n). Auxiliary space: O(1). A brute-force solution recalculates each k-element sum, taking O(nk) time. Sliding window reuses the previous sum, so each move needs only one subtraction and one addition.

Edge case: initialise the best sum with the first window's actual sum, not zero. If all values are negative, starting at zero would give an incorrect answer.

4. Variable-size sliding window

A variable-size window changes its length. The right pointer expands the window by including new elements. When the current window satisfies the condition, the left pointer moves right to make it smaller while preserving the condition.

General pattern for a positive-sum threshold

```
left = 0
sum = 0
bestLength = infinity

for right from 0 to n - 1:
    sum += arr[right]

    while sum >= target:
        bestLength = min(bestLength, right - left + 1)
        sum -= arr[left]
        left++
```

This is suitable for finding the shortest subarray with sum at least a target when all values are positive (and also works for non-negative values with care). With negative values, removing the left element may increase the sum, so the simple expand/shrink reasoning no longer holds.

Why use while instead of if?

One right-side addition can make the window larger than necessary. The while loop repeatedly removes elements from the left as long as the condition remains true. That way, the algorithm considers the shortest valid window ending at each right boundary.

Problem 2 — Minimum length subarray with sum at least target

Problem: Given an array of positive integers and a target, return the minimum length of a contiguous subarray whose sum is greater than or equal to the target. Return 0 if no such subarray exists.

Example input: target = 7, arr = [2, 3, 1, 2, 4, 3]
Expected output: 2

Reasoning

The window [2, 3, 1, 2] has sum 8 and length 4. Shrinking from the left by removing 2 leaves sum 6, which is too small. Later, [4, 3] has sum 7 and length 2, so the best answer is 2. No single element reaches 7.

5. Problem 2 — Java solution and trace

Complete Java program

```
public class MinimumSubarrayLength {
    static int minSubArrayLen(int target, int[] nums) {
        int left = 0;
        long sum = 0;
        int minLength = Integer.MAX_VALUE;

        for (int right = 0; right < nums.length; right++) {
            sum += nums[right];

            while (sum >= target) {
                int length = right - left + 1;
                minLength = Math.min(minLength, length);
                sum -= nums[left];
                left++;
            }
        }

        return minLength == Integer.MAX_VALUE ? 0 : minLength;
    }

    public static void main(String[] args) {
        int target = 7;
        int[] nums = {2, 3, 1, 2, 4, 3};
        System.out.println(minSubArrayLen(target, nums));
    }
}
```

Output: 2

Trace the important steps

Right value added	Current window / sum	Action
2	[2], sum = 2	Below target
3	[2, 3], sum = 5	Below target
1	[2, 3, 1], sum = 6	Below target
2	[2, 3, 1, 2], sum = 8	Valid; record length 4, remove 2; sum = 6
4	[3, 1, 2, 4], sum = 10	Record valid windows while shrinking
3	[4, 3], sum = 7	Record length 2; best answer is 2

Time complexity is O(n): the right pointer moves forward n times, and the left pointer also moves forward at most n times. Although there is a while loop inside the for loop, total left-pointer moves are bounded by n. Extra space is O(1).

This solution assumes positive values. If negative numbers are allowed, a different technique may be needed, such as prefix sums with a monotonic deque, depending on the exact problem.

6. Choosing the right window + revision

Fixed vs. variable window

Question type	Likely technique
Largest sum among exactly k consecutive elements	Fixed-size window
Longest substring without repeated characters	Variable window + character tracking
Shortest positive-number subarray with sum ≥ target	Variable window
Maximum value in every k-element window	Window + deque
Choose any elements, not necessarily consecutive	Usually not sliding window

Common mistakes

• Forgetting to validate k for a fixed-size window.
• Recomputing the whole window sum after every move and losing the efficiency gain.
• Using if instead of while when a variable window must shrink repeatedly.
• Updating the best answer before the current window satisfies the condition.
• Applying a sum-based variable window to arrays with negative values without checking whether the logic remains valid.

Practice questions

1. Find the average of every contiguous subarray of size k.
2. Find the longest substring without repeating characters.
3. Find the maximum value in each window of size k.
4. Count how many windows of size k have a sum greater than a given threshold.

Final takeaway

Sliding window is useful when the problem concerns a continuous range and neighbouring ranges overlap. For a fixed window, update the state by removing what leaves and adding what enters. For a variable window, move the right boundary to expand and the left boundary to restore the condition. Explain why each pointer only moves forward; that is the core reason many sliding-window solutions run in linear time.

Study habit: dry-run both programs with a small input, then change the array values and verify the output yourself.